

Part 1 — Crisis Assessment & Immediate Stabilization Plan

Executive Summary

The enterprise platform, processing **\$50 million in daily transactions**, is facing a critical operational crisis. With production downtime costing over **\$100,000 per hour**, the current 70% deployment failure rate poses an existential threat to business continuity and regulatory compliance. The inability to reliably deploy new features undermines weekly release mandates and erodes customer trust.

This document outlines a 30-day emergency stabilization plan to achieve a **deployment success rate exceeding 90%** and reduce rollback times to under **15 minutes**. The scope encompasses the entire platform ecosystem: **12 microservices** and **3 microfrontends**. Our strategy is to immediately halt cascading failures, establish automated and consistent deployment patterns, and build a visible, controllable release process.

The plan progresses from immediate firefighting in Week 1 to foundational automation in Weeks 2-3, culminating in advanced orchestration and observability by Day 30. This phased approach ensures we deliver rapid stability improvements while laying the groundwork for a modern, secure DevSecOps pipeline.

Root Cause Analysis

An analysis of the current deployment process and the provided sample repositories reveals five systemic root causes for the 70% failure rate:

1. Hardcoded Secrets in Dockerfiles and K8s Manifests

- **Symptoms:** ENV DB_PASSWORD=... in Dockerfiles; plaintext secrets in kubectl configs.
- **Risk & Impact:** Causes immediate runtime failures when dev configurations are deployed to production. Represents a critical security breach point, violating compliance and exposing sensitive data.

2. No Health Checks in Kubernetes Manifests

- **Symptoms:** Missing readinessProbe and livenessProbe in all sample service deployments.
- **Risk & Impact:** Kubernetes routes traffic to unhealthy pods. This turns a single service failure into a cascading outage, as dependent services (e.g., payment-service → account-service) fail in sequence.

3. Manual Database Migrations Without Safeguards

- **Symptoms:** Migrations executed via ad-hoc SQL scripts against production databases.
- **Risk & Impact:** High risk of schema-version mismatch causing application failure. The lack of snapshots or dry-run validation creates a direct path to irreversible data loss.

4. Manual, Inconsistent Docker Builds and Tags

- **Symptoms:** Use of docker build -t service:latest on individual machines.

- **Risk & Impact:** The latest tag creates artifact drift. The application tested in a development environment is not the same as the one running in production, leading to unpredictable and undebuggable failures.

5. No Automated Testing and Dependency Coordination

- **Symptoms:** Build command `mvn clean package -DskipTests`; no orchestration for deploying interdependent services.
- **Risk & Impact:** Untested code and bugs reach production. Deploying services like payment-service without ensuring account-service is ready causes cascading system-wide failures.

Prioritization & Timeline

Immediate (Days 0-7): Stop the Bleeding

- Standardize Docker builds using CI/CD (GitHub Actions).
- Add Kubernetes readiness/liveness probes to all services.
- Externalize all configurations to ConfigMaps and Secrets (HashiCorp Vault).

Short-Term (Days 7-21): Build Foundations

- Enforce automated testing (unit, integration) in the CI pipeline.
- Integrate container image scanning (Trivy/Snyk) and SAST.
- Automate database migrations using Flyway within the deployment pipeline.

30-Day Target (Days 21-30): Advanced Control & Visibility

- Implement GitOps with ArgoCD for declarative deployments.
- Finalize a robust ≤ 15 -minute rollback strategy.
- Deploy full observability stack (Prometheus/Grafana, EFK/ELK).

30-Day Emergency Plan: Step-by-Step Actions

Week 1: Critical Stabilization

- **Day 1-2:** Implement basic health checks for all 12 services in Kubernetes.
- **Day 3-4:** Configure centralized secret management; remove all hardcoded values.
- **Day 5-7:** Establish automated CI pipeline for 3 core services (payment, account, transaction) with standardized build and tag.

Week 2-3: Automation & Quality Gates

- **Day 8-10:** Integrate unit and integration tests as mandatory pipeline gates.
- **Day 11-14:** Implement automated, version-controlled database migrations for core services.
- **Day 15-18:** Deploy centralized logging (Fluentd -> Elasticsearch) for basic visibility.
- **Day 19-21:** Introduce security scanning (SAST/DAST) and image vulnerability checks.

Week 4: Orchestration & Enforcement

- **Day 22-24:** Implement ArgoCD for GitOps-style deployment to development and staging.
- **Day 25-26:** Conduct rollback drills and finalize the 15-minute rollback runbook.
- **Day 27-28:** Team training sessions on new pipelines and runbooks.
- **Day 29-30:** Go-Live for new deployment process with manual approval gate for production.

Rollback Strategy (≤15 minutes)

1. **Application Rollback:**
2. # For Helm-based deployments
3. helm rollback <release-name> <previous-revision>
- 4.
5. # For ArgoCD/GitOps

git revert HEAD # ArgoCD automatically syncs the previous state

6. **Database Rollback Safeguards:**
 - Take a pre-migration snapshot of the RDS database.
 - Maintain forward-only, idempotent Flyway migration scripts with a dedicated rollback script folder.
7. **Infrastructure Failover:**
 - Configure fast-route DNS failover to a standby region if needed.
8. **Runbook Ownership:**
 - Designate a primary and secondary on-call engineer responsible for executing the rollback runbook during deployments.

Risks & Mitigations

Risk Category	Mitigation Steps
Security & Compliance	- Mandatory secret management from Day 1. - All changes tracked in Git for audit trails. - Maintain manual approval gates for production during stabilization.
Migration & Stability	- Apply all changes to development and UAT first. - Conduct failover drills on the Disaster Recovery (DR) environment weekly. - Implement feature flags to disable new features instantly.
Team Skill Gap	- Create detailed, step-by-step runbooks. - Implement a "pair-deployment" model for the first two weeks.

Team Training & Readiness

- **Runbook Drills:** Conduct mandatory rollback drills twice in Week 3.
- **1:1 Sessions:** Senior engineers hold sessions on IaC, Kubernetes, and new tooling.
- **On-Call Designation:** Establish a clear, rotating on-call schedule with primary/secondary roles defined.
- **Documentation Hub:** Maintain a single source of truth for all runbooks and architecture diagrams.